

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2021 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Model 2021	2
SUBIECTUL I (30 de puncte)	2
1. Grafuri și liste de adiacență	2
2. Procesoare de text: Inserare ilustrații și diagrame	2
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Lista alternantă paritate	3
2. Algoritm eficient: Intervale disjuncte (ISMP)	4
II. Rezolvare Titularizare Varianta 2 (21 iulie 2021)	6
SUBIECTUL I	6
1. Partiții ale unei mulțimi	6
2. TIC: Grafice și ilustrații într-un document	7
SUBIECTUL al II-lea	8
1. Programare: Adăugare în listă alternantă	8
2. Algoritm eficient: Intervale disjuncte	9
SUBIECTUL al III-lea (30 de puncte)	11
1. Test pentru secvența A: minim/maxim în secvențe	11
2. Lucru pe grupe pentru secvența B: E-mail și FTP	12

I. Rezolvare Titularizare Model 2021

SUBIECTUL I (30 de puncte)

1. Grafuri și liste de adiacență

- **Graf neorientat:** Definiție, gradele nodurilor, număr de muchii ($\sum d(v) = 2M$).
- **Reprezentare:** Matrice de adiacență vs liste de adiacență.
- **Problemă (Gradul maxim în graf):**

► C++:

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    int n, m, u, v;
    cin >> n >> m;
    vector<int> grad(n + 1, 0);
    for (int i = 0; i < m; ++i) {
        cin >> u >> v;
        grad[u]++; grad[v]++;
    }
    int max_g = 0;
    for (int i = 1; i <= n; ++i) if (grad[i] > max_g) max_g = grad[i];
    cout << max_g << endl;
    return 0;
}
```

► Pascal:

```
program GradMaxim;
var
    n, m, u, v, i, max_g: integer;
    grad: array[1..100] of integer;
begin
    read(n, m);
    for i := 1 to n do grad[i] := 0;
    for i := 1 to m do
        begin
            read(u, v);
            grad[u] := grad[u] + 1;
            grad[v] := grad[v] + 1;
        end;
    max_g := 0;
    for i := 1 to n do if grad[i] > max_g then max_g := grad[i];
    writeln(max_g);
end.
```

2. Procesoare de text: Inserare ilustrații și diagrame

- **Tipuri:** Imagini (PNG/JPG), Forme geometrice (Shapes), Diagrame SmartArt.
- **Personalizare:** Culori de fundal, efecte 3D, umbre, aliniere text în jurul imaginii (Wrap Text), încadrare în pagină.

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Lista alternantă paritate

Soluție C++:

```
#include <iostream>
using namespace std;

struct Nod {
    int info;
    Nod* urm;
};

void adaug(Nod* &p, Nod* &u, int x) {
    Nod* nou = new Nod{x, nullptr};
    if (p == nullptr) {
        p = nou;
        u = nou;
    } else {
        u->urm = nou;
        u = nou;
    }
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Nod *p = nullptr, *u = nullptr;
    int odd = 1;
    int even = n;

    for (int i = 0; i < n / 2; ++i) {
        adaug(p, u, odd);
        adaug(p, u, even);
        odd += 2;
        even -= 2;
    }

    for (Nod* curr = p; curr != nullptr; curr = curr->urm) {
        cout << curr->info << (curr->urm == nullptr ? "" : " ");
    }
    cout << endl;

    return 0;
}
```

Soluție Pascal:

```
program ListaAlternanta;
type
    PNod = ^TNod;
    TNod = record
        info: integer;
        urm: PNod;
    end;
var
```

```

p, u, curr: PNode;
n, i, odd, even: integer;

procedure adaug(var p, u: PNode; x: integer);
var nou: PNode;
begin
    new(nou);
    nou^.info := x;
    nou^.urm := nil;
    if p = nil then
    begin
        p := nou;
        u := nou;
    end
    else
    begin
        u^.urm := nou;
        u := nou;
    end;
end;

begin
    readln(n);
    p := nil;
    u := nil;
    odd := 1;
    even := n;

    for i := 1 to n div 2 do
    begin
        adaug(p, u, odd);
        adaug(p, u, even);
        odd := odd + 2;
        even := even - 2;
    end;

    curr := p;
    while curr <> nil do
    begin
        write(curr^.info);
        if curr^.urm <> nil then write(' ');
        curr := curr^.urm;
    end;
    writeln;
end.

```

2. Algoritm eficient: Intervale disjuncte (ISMP)

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

```

```

struct Interval {
    long long st, dr;
};

bool cmp(const Interval &a, const Interval &b) {
    if (a.dr != b.dr) return a.dr < b.dr;
    return a.st < b.st;
}

int main() {
    ifstream fin("titu2021.in");
    int n;
    if (!(fin >> n)) return 0;
    vector<Interval> intervals(n);
    for (int i = 0; i < n; ++i) {
        fin >> intervals[i].st >> intervals[i].dr;
    }
    fin.close();

    sort(intervals.begin(), intervals.end(), cmp);

    int count = 0;
    long long last_dr = -2e18;
    for (int i = 0; i < n; ++i) {
        if (intervals[i].st >= last_dr) {
            count++;
            last_dr = intervals[i].dr;
        }
    }

    cout << count << endl;
    return 0;
}

```

Soluție Pascal:

```

program IntervaleDisjuncte;
type
    Interval = record
        st, dr: int64;
    end;
var
    fin: text;
    intervals: array[1..10000] of Interval;
    n, i, count: integer;
    last_dr: int64;

procedure QuickSort(l, r: integer);
var
    i_idx, j_idx: integer;
    pivot, tmp: Interval;
begin
    i_idx := l; j_idx := r;
    pivot := intervals[(l + r) div 2];
    repeat
        while (intervals[i_idx].dr < pivot.dr) or ((intervals[i_idx].dr = pivot.dr) and
(intervals[i_idx].st < pivot.st)) do i_idx := i_idx + 1;

```

```

    while (intervals[j_idx].dr > pivot.dr) or ((intervals[j_idx].dr = pivot.dr) and
(intervals[j_idx].st > pivot.st)) do j_idx := j_idx - 1;
    if i_idx <= j_idx then
    begin
        tmp := intervals[i_idx]; intervals[i_idx] := intervals[j_idx];
intervals[j_idx] := tmp;
        i_idx := i_idx + 1; j_idx := j_idx - 1;
    end;
    until i_idx > j_idx;
    if l < j_idx then QuickSort(l, j_idx);
    if i_idx < r then QuickSort(i_idx, r);
end;

begin
    assign(fin, 'titu2021.in'); reset(fin);
    read(fin, n);
    for i := 1 to n do read(fin, intervals[i].st, intervals[i].dr);
    close(fin);

    QuickSort(1, n);

    count := 0;
    last_dr := -9999999999999999;
    for i := 1 to n do
    begin
        if intervals[i].st >= last_dr then
        begin
            count := count + 1;
            last_dr := intervals[i].dr;
        end;
    end;
    writeln(count);
end.

```

II. Rezolvare Titularizare Varianta 2 (21 iulie 2021)

[Subiect PDF](file:

SUBIECTUL I

1. Partiții ale unei mulțimi

O partiție a unei mulțimi cu n elemente este o descompunere în submulțimi nevide, disjuncte două câte două, a căror reuniune este mulțimea inițială. O metodă standard de generare folosește un vector $p[1..n]$, unde $p[i]$ reprezintă blocul în care este așezat elementul i .

Algoritmul construiește recursiv vectorul: pentru elementul curent k , se încearcă blocurile deja existente $1..nrBlocuri$ și, suplimentar, un bloc nou $nrBlocuri+1$. La final, când $k=n+1$, vectorul descrie o partiție validă. Pentru a evita duplicarea, blocurile sunt introduse în ordinea apariției lor. Complexitatea este proporțională cu numărul partițiilor generate, adică numărul Bell B_n , iar memoria auxiliară este $O(n)$.

Exemplu pentru $\{1, 2, 3, 4\}$: primele partiții generate pot fi $\{1, 2, 3, 4\}$, $\{1, 2, 3\}\{4\}$, $\{1, 2, 4\}\{3\}$, $\{1, 2\}\{3, 4\}$, $\{1, 2\}\{3\}\{4\}$, $\{1, 3, 4\}\{2\}$ etc. Pentru $n=4$ există $B_4 = 15$ partiții.

Problemă: Se citește n . Să se afișeze toate partițiile mulțimii $\{1, 2, \dots, n\}$.

Soluție C++:

```
#include <iostream>
#include <vector>
using namespace std;

int n, p[20];

void afisare(int nrBlocuri) {
    for (int b = 1; b <= nrBlocuri; ++b) {
        cout << "{";
        bool primul = true;
        for (int i = 1; i <= n; ++i) {
            if (p[i] == b) {
                if (!primul) cout << ",";
                cout << i;
                primul = false;
            }
        }
        cout << "}";
    }
    cout << "\n";
}

void gen(int k, int nrBlocuri) {
    if (k > n) {
        afisare(nrBlocuri);
        return;
    }
    for (int b = 1; b <= nrBlocuri; ++b) {
        p[k] = b;
        gen(k + 1, nrBlocuri);
    }
    p[k] = nrBlocuri + 1;
    gen(k + 1, nrBlocuri + 1);
}

int main() {
    cin >> n;
    p[1] = 1;
    gen(2, 1);
    return 0;
}
```

Descrierea soluției: Elementul 1 este pus în primul bloc pentru a fixa o reprezentare unică. Pentru fiecare element următor, fie îl plasăm într-un bloc existent, fie deschidem un bloc nou. Astfel se generează toate partițiile fără permutări duplicate ale blocurilor.

2. TIC: Grafice și ilustrații într-un document

Într-un procesor de text, ilustrațiile pot fi imagini, forme automate, diagrame, capturi de ecran, SmartArt sau grafice. Inserarea se face din meniul dedicat obiectelor grafice, apoi obiectul se poate redimensiona, decupa, roti, alinia, ancora față de paragraf și formata prin bordură, umbră, contrast sau stil.

Pentru un grafic statistic, datele se introduc într-un tabel asociat, se alege tipul adecvat (coloane, linii, sectoare), se completează titlul, legenda și etichetele axelor, apoi se verifică lizibilitatea. O ilustrație corect folosită sprijină înțelegerea textului, nu îl înlocuiește și trebuie să fie poziționată coerent cu paragraful la care se referă.

SUBIECTUL al II-lea

1. Programare: Adăugare în listă alternantă

Soluție C++:

```
#include <iostream>
using namespace std;

struct Nod {
    int info;
    Nod* urm;
};

void adaug(Nod* &p, Nod* &u, int x) {
    Nod* nou = new Nod{x, nullptr};
    if (p == nullptr) {
        p = nou;
        u = nou;
    } else {
        u->urm = nou;
        u = nou;
    }
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    Nod *p = nullptr, *u = nullptr;
    int odd = 1;
    int even = n;

    for (int i = 0; i < n / 2; ++i) {
        adaug(p, u, odd);
        adaug(p, u, even);
        odd += 2;
        even -= 2;
    }

    for (Nod* curr = p; curr != nullptr; curr = curr->urm) {
        cout << curr->info << (curr->urm == nullptr ? "" : " ");
    }
    cout << endl;

    return 0;
}
```

Soluție Pascal:

```
program ListaAlternanta;
type
    PNod = ^TNod;
```



```

TNod = record
    info: integer;
    urm: PNod;
end;
var
    p, u, curr: PNod;
    n, i, odd, even: integer;

procedure adaug(var p, u: PNod; x: integer);
var nou: PNod;
begin
    new(nou);
    nou^.info := x;
    nou^.urm := nil;
    if p = nil then
    begin
        p := nou;
        u := nou;
    end
    else
    begin
        u^.urm := nou;
        u := nou;
    end;
end;

begin
    readln(n);
    p := nil;
    u := nil;
    odd := 1;
    even := n;

    for i := 1 to n div 2 do
    begin
        adaug(p, u, odd);
        adaug(p, u, even);
        odd := odd + 2;
        even := even - 2;
    end;

    curr := p;
    while curr <> nil do
    begin
        write(curr^.info);
        if curr^.urm <> nil then write(' ');
        curr := curr^.urm;
    end;
    writeln;
end.

```

2. Algoritm eficient: Intervale disjuncte

Metoda: Problema este una de selecție maximă a intervalelor compatibile. Sortăm intervalele crescător după extremitatea dreaptă și alegem greedy primul interval care se termină cel mai

devreme. Apoi selectăm următorul interval al cărui capăt stâng este cel puțin capătul drept al ultimului interval selectat. Pentru intervale deschise, (a,b) și (b,c) au intersecția vidă, deoarece punctul b nu aparține niciunuia dintre intervale, deci condiția $st \geq last_dr$ este corectă.

Eficiență: Sortarea are complexitatea $O(n \log n)$, iar parcurgerea greedy este $O(n)$. Memoria este $O(n)$ pentru memorarea intervalelor.

Soluție C++:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <algorithm>
using namespace std;

struct Interval {
    long long st, dr;
};

bool cmp(const Interval &a, const Interval &b) {
    if (a.dr != b.dr) return a.dr < b.dr;
    return a.st < b.st;
}

int main() {
    ifstream fin("titu2021.in");
    int n;
    if (!(fin >> n)) return 0;
    vector<Interval> intervals(n);
    for (int i = 0; i < n; ++i) {
        fin >> intervals[i].st >> intervals[i].dr;
    }
    fin.close();

    sort(intervals.begin(), intervals.end(), cmp);

    int count = 0;
    long long last_dr = -2e18;
    for (int i = 0; i < n; ++i) {
        if (intervals[i].st >= last_dr) {
            count++;
            last_dr = intervals[i].dr;
        }
    }

    cout << count << endl;
    return 0;
}
```

Soluție Pascal:

```
program IntervaleDisjuncte;
type
    Interval = record
        st, dr: int64;
    end;
```

```

var
  fin: text;
  intervals: array[1..10000] of Interval;
  n, i, count: integer;
  last_dr: int64;

procedure QuickSort(l, r: integer);
var
  i_idx, j_idx: integer;
  pivot, tmp: Interval;
begin
  i_idx := l; j_idx := r;
  pivot := intervals[(l + r) div 2];
  repeat
    while (intervals[i_idx].dr < pivot.dr) or ((intervals[i_idx].dr = pivot.dr) and
(intervals[i_idx].st < pivot.st)) do i_idx := i_idx + 1;
    while (intervals[j_idx].dr > pivot.dr) or ((intervals[j_idx].dr = pivot.dr) and
(intervals[j_idx].st > pivot.st)) do j_idx := j_idx - 1;
    if i_idx <= j_idx then
      begin
        tmp := intervals[i_idx]; intervals[i_idx] := intervals[j_idx];
        intervals[j_idx] := tmp;
        i_idx := i_idx + 1; j_idx := j_idx - 1;
      end;
    until i_idx > j_idx;
    if l < j_idx then QuickSort(l, j_idx);
    if i_idx < r then QuickSort(i_idx, r);
  end;

begin
  assign(fin, 'titu2021.in'); reset(fin);
  read(fin, n);
  for i := 1 to n do read(fin, intervals[i].st, intervals[i].dr);
  close(fin);

  QuickSort(1, n);

  count := 0;
  last_dr := -9999999999999999;
  for i := 1 to n do
    begin
      if intervals[i].st >= last_dr then
        begin
          count := count + 1;
          last_dr := intervals[i].dr;
        end;
    end;
  writeln(count);
end.

```

SUBIECTUL al III-lea (30 de puncte)

1. Test pentru secvența A: minim/maxim în secvențe

Testul are trei itemi, 90 de puncte și 10 puncte din oficiu.

1. **Item obiectiv (20p):** Maximul secvenței 4 9 2 este: A. 2 B. 4 C. 9 D. 15. **Răspuns:** C.
2. **Item semiobiectiv (30p):** Completați pseudocodul pentru actualizarea minimului când $x < \text{minim}$: `minim <- [spațiu liber]`. **Răspuns:** x.
3. **Problemă (40p):** Scrieți algoritmul care citește n valori și afișează minimul și maximul.
Barem: citire 10p, inițializare 10p, parcurgere/teste 15p, afișare 5p.

2. Lucru pe grupe pentru secvența B: E-mail și FTP

Caracteristici: responsabilitate comună și interacțiune între elevi. **Argument:** serviciile Internet se înțeleg bine prin sarcini distribuite.

Metodă: mozaic. **Mijloc:** fișe și calculator. **Activitate:** compararea serviciilor E-mail și FTP.

Scenariu: Grupele studiază câte un serviciu, identifică rol, avantaje, riscuri, apoi prezintă concluziile și completează un tabel comparativ.